УЧЕБНЫЙ ПРОЕКТ / PYTHON

PARALLEL SORT LAB

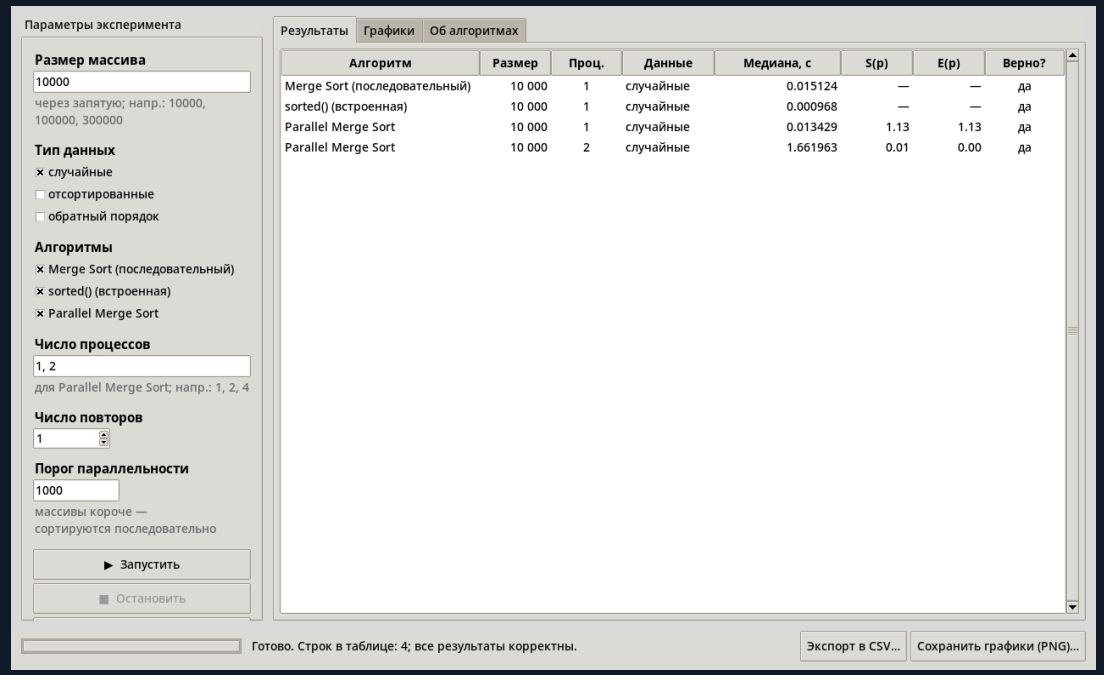
Параллельные алгоритмы сортировки

Сравнение Merge Sort, Parallel Merge Sort и sorted()

через измерения, метрики и визуализацию

5 УЧАСТНИКОВ

TKINTER + MATPLOTLIB



ОТ ИДЕИ — К ИЗМЕРЕНИЮ

Код • реальные тесты • CSV • графики • отчёт

Сдача: 29 октября 2026

Три реализации — одна задача

Сортируем одинаковые массивы, измеряем полное время вызова и проверяем результат.

БАЗОВЫЙ АЛГОРИТМ

Merge Sort

Собственная последовательная реализация.

O(n log n) по времени, O(n) дополнительной памяти.

ЧТО ИССЛЕДУЕМ

Parallel Merge Sort

Разбиение на p частей → отдельные процессы → слияние в главном процессе.

ОРИЕНТИР

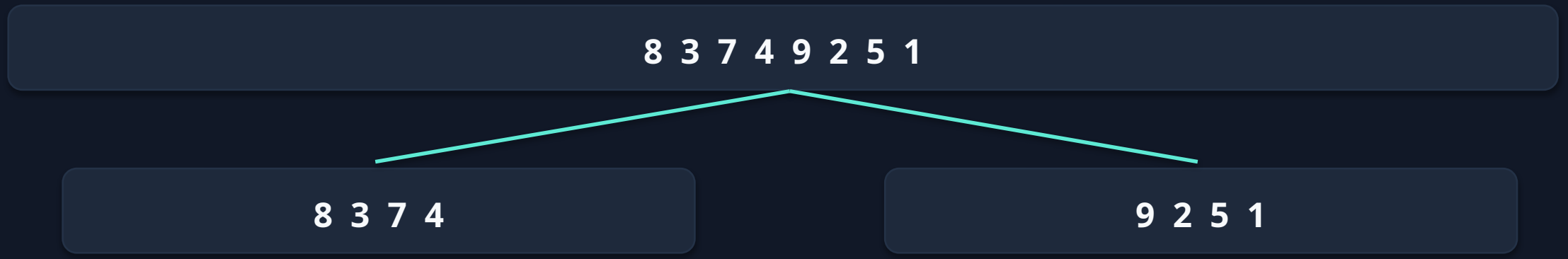
sorted()

Встроенный высокооптимизированный Timsort. Практический эталон, а не наш алгоритм.

Критерий корректности — **Каждый результат должен совпадать с sorted(исходный_массив)**

Merge Sort: «разделяй и властвуй»

Рекурсивное деление уменьшает размер задачи; слияние объединяет отсортированные части линейно.



Деление

до массивов длины 1

Рекурсивная сортировка

каждая половина сортируется отдельно

Слияние

два указателя, линейный проход

Результат: 1 2 3 4 5 7 8 9

Сложность: что гарантирует алгоритм

Скорость растёт не только от алгоритма, но и от реализации и накладных расходов среды.

ВРЕМЯ

O(n log n)

Временная сложность Merge Sort в лучшем, среднем и худшем случаях.

ПАМЯТЬ

O(n)

Дополнительная память для временных массивов при слиянии.

СВОЙСТВО

Устойчивость

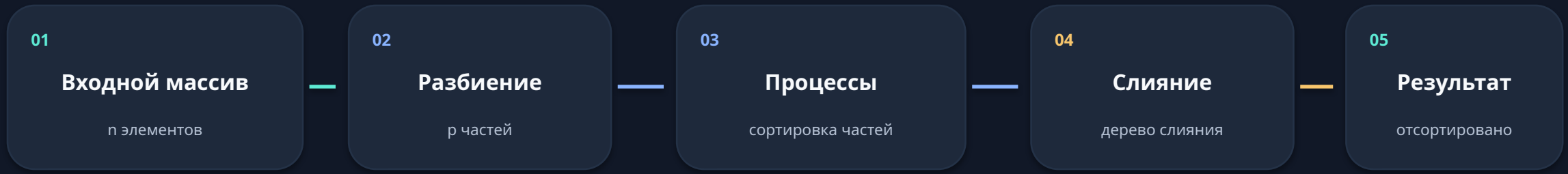
Равные элементы сохраняют порядок, если при равенстве брать левую часть.

$$T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$$

Глубина рекурсии — $\log_2(n)$; суммарная работа каждого уровня слияния — $O(n)$.

Как работает Parallel Merge Sort

Один пул процессов на вызов; финальное слияние в текущей реализации остаётся последовательным.



Число процессов

workers = 1, 2, 4

Порог

малые массивы — последовательно

Платформа

spawn совместим с macOS

Параллелизм не бесплатный

Процессы дают возможность выполнять вычисления параллельно, но добавляют свои расходы.

01 **Запуск**

Создание процессов и их жизненный цикл.

02 **Передача**

Сериализация списков и передача данных через pickle.

03 **Слияние**

Объединение частей выполняется в главном процессе.

04 **Ресурсы**

Если процессов больше CPU, они конкурируют за время процессора.

По закону Амдала последовательная часть ограничивает теоретический выигрыш: $S(p) \le 1 / (f + (1 - f) / p)$.

Сравнение без подмены условий

Одинаковые данные, таймер высокой точности и проверка результата вне измеряемого времени.

1

Сгенерировать

Массив генерируется один раз.

2

Измерить

time.perf_counter(); полный вызов сортировки.

3

Повторить

3 повтора, итог — медиана.

4

Проверить

Сравнить с sorted(вход).

5

Сохранить

CSV + графики времени, S и E.

Параметры

random: 10k / 50k / 100k • sorted/reversed: 50k • 3 повтора • p = 1 / 2 / 4

S(p) = T_merge / T_parallel(p) | E(p) = S(p) / p

Что показали реальные измерения

Медиана времени, миллисекунды. Случайные данные, по 3 повтора; все результаты корректны.

Размер n	Merge Sort	sorted()	Parallel p=2	Parallel p=4
10 000	14.17	1.02	921.60	1684.18
50 000	82.61	6.13	733.87	1469.59
100 000	172.23	13.35	886.07	1611.87

sorted() — быстрее

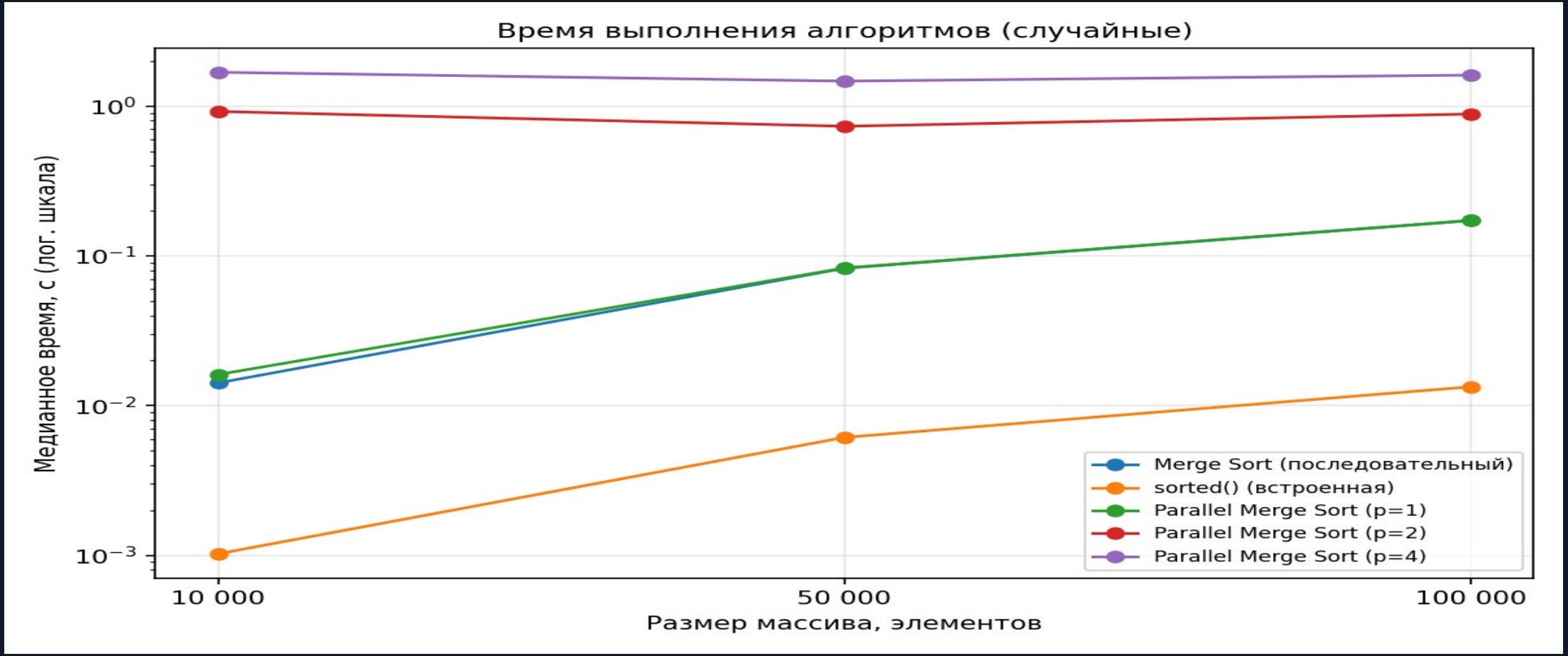
При n = 100 000: 13,35 мс против 172,23 мс у Merge Sort.

p = 2 и p = 4 — медленнее

Пул и передача данных доминируют на протестированных размерах.

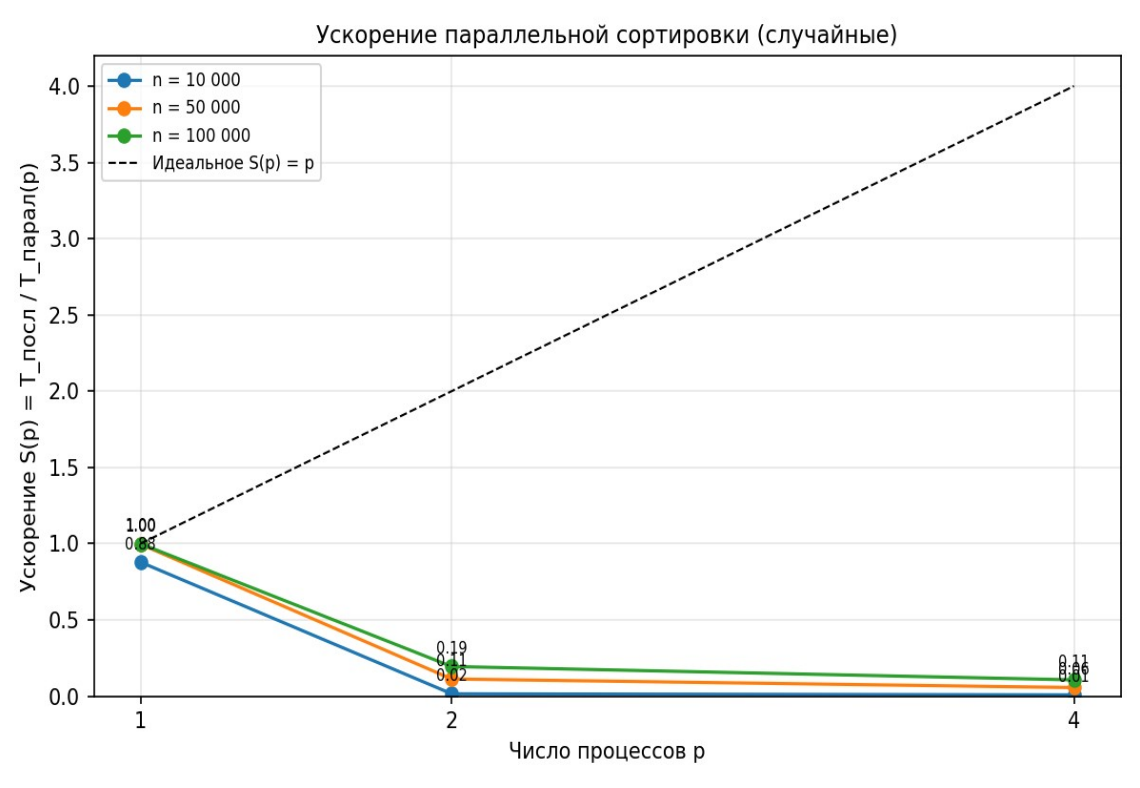
Время выполнения

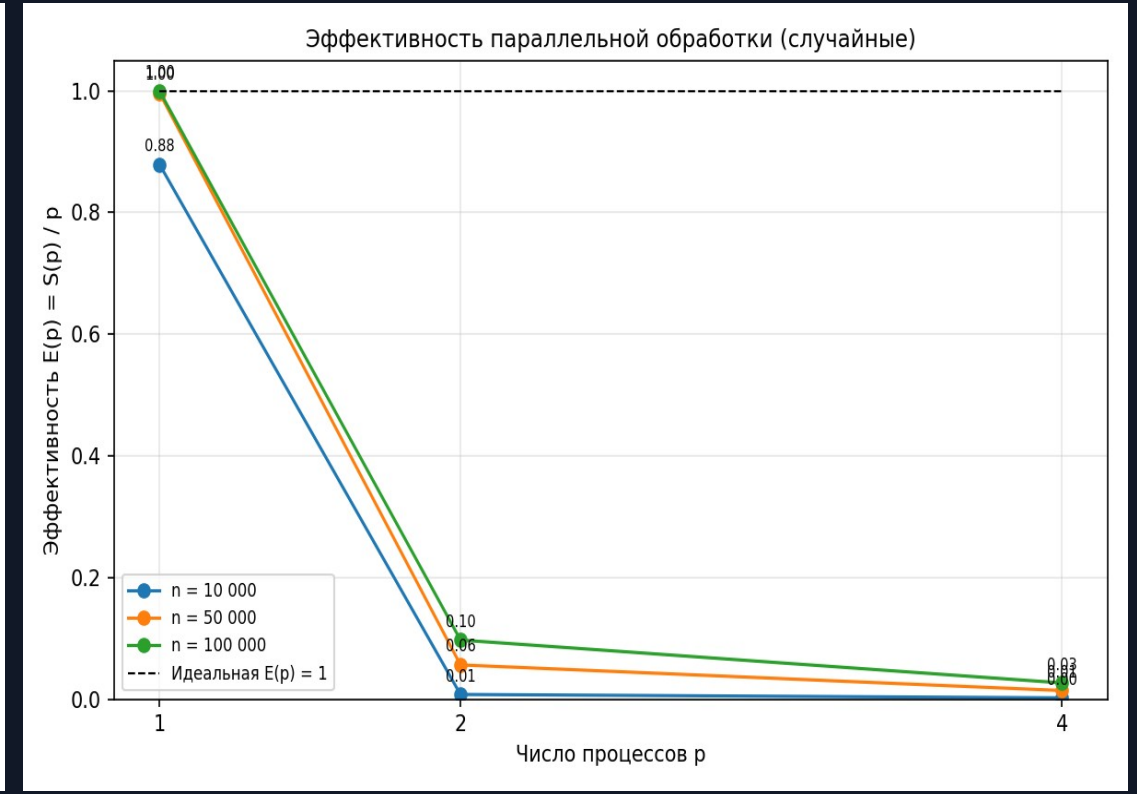
Логарифмическая шкала показывает разницу между Python Merge Sort, sorted() и вызовом с новыми процессами.



Ускорение оказалось меньше единицы

Для данной реализации и стенда дополнительный процесс увеличил полное время вызова.

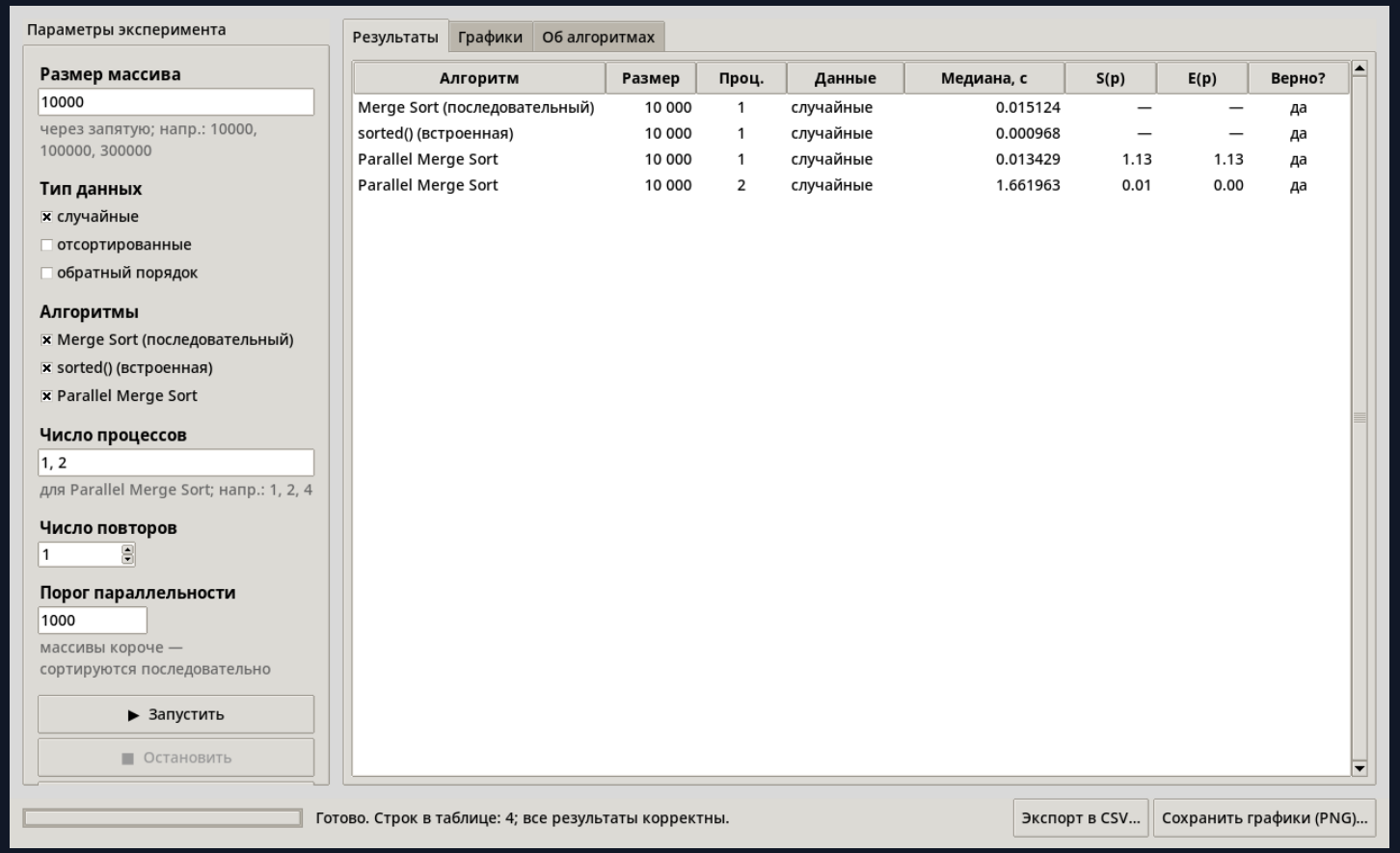




При n = 100 000: S(2) = 0.194; E(2) = 0.097. Значение S < 1 означает замедление, а не ускорение.

Интерфейс, тесты и экспорт

Пользователь задаёт эксперимент, видит таблицу, графики, корректность и выгружает CSV.



Запуск: python3 main.py • Тесты: xvfb-run -a python3 -m unittest discover -s tests -v (Linux)

32 автоматических теста

Запуск через unittest; GUI проверен в виртуальном дисплее.

CSV + 3 графика

Время, ускорение и эффективность; графики строятся из измерений.

25 строк результатов

Все выходы эксперимента совпали с sorted(вход).

Результат исследования

Параллельность — инструмент, а не гарантия ускорения.

1. Корректность

Все 25 итоговых строк CSV содержат корректный результат.

2. Накладные расходы

Запуск процессов и передача данных доминировали на выбранных размерах.

3. Следующий шаг

Повторить тест на целевом Mac и изучить переиспользование пула.

РАСПРЕДЕЛЕНИЕ ЗАЩИТЫ

У1 — архитектура и итоги • У2 — теория и Merge Sort • У3 — параллельная версия
У4 — эксперимент и графики • У5 — интерфейс, тесты и инструкция запуска

Ограничение: измерения выполнены в Linux-среде с 3 логическими CPU; значения зависят от оборудования.